

EE332 数字系统设计

Lab 3-2: 全加器设计

姓 名: 何衍宁
学 号: 12311512

2026 年 3 月 12 日

目 录

1	实验目的	2
2	VHDL 代码和 testbench 代码	2
3	结果	4
3.1	behavior simulation	4
3.2	post synthesis timing simulation	5
3.3	post implementation timing simulation	6
4	分析 (Q&A)	6
4.1	Q1. 仿真结果是否与上页图中的结果一致?	6
4.2	Q2. 加法计算结果是否和人们通常的认知一致?	6
4.3	Q3. 如不一致, 导致不一致的原因是什么?	6
4.4	Q4. 3 种仿真结果有什么不同?	7
4.5	Q5. 为什么会有不同?	7
4.6	Q6. 你是否在结果看了一些尖峰?	7
4.7	Q7. 这些尖峰是由什么引起的?	8
5	结论	9

1 实验目的

本实验主要进行了全加器的设计、仿真与分析过程。通过使用 VHDL 语言编写全加器的实现代码，并编写相应的 Testbench 代码，对该设计进行了功能验证。随后，通过分析不同仿真阶段的结果，探讨了仿真结果与理论预期之间的差异及其产生的原因。

2 VHDL 代码和 testbench 代码

全加器采用代码如下：

```
1 library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3
4 entity full_adder1 is
5 Port ( A,B,C_in : in STD_LOGIC;
6       Sum,C_out : out STD_LOGIC);
7 end full_adder1;
8
9 architecture dataflow of full_adder1 is
10 -- 内部信号
11 signal s1, s2, s3: STD_LOGIC;
12 constant gate_delay : time := 10ns;
13 begin
14 -- 求和
15 L1:s1 <= (A xor B) after gate_delay;
16 -- 进位
17 L2:s2 <= (C_in and s1) after gate_delay;
18 -- 进位
19 L3:s3 <= (A and B) after gate_delay;
20 -- 本位
21 L4:Sum <= (s1 xor C_in) after gate_delay;
22 -- 进位相加
23 L5:C_out <= (s2 or s3) after gate_delay;
24 end dataflow;
```

为验证全加器采用的代码是否正确，输入如下波形：

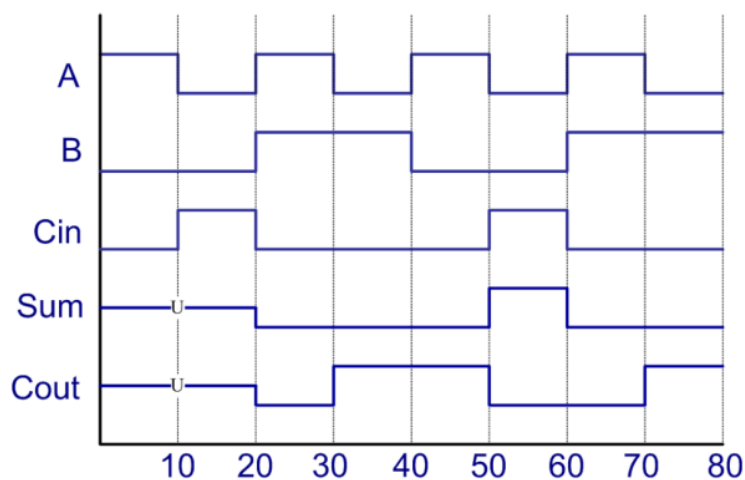


图 1: Origin Wave

编辑 testbench 的代码如下:

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3
4  entity testbench is
5  end testbench;
6
7  architecture tb of testbench is
8  component full_adder1 is
9      Port( A,B,C_in : in STD_LOGIC;
10          Sum,C_out : out STD_LOGIC);
11 end component;
12
13 signal a,b,ci,s,co : STD_LOGIC;
14 constant period : time := 10ns;
15
16 begin
17     uut: full_adder1 port
18         map( A => a,
19             B => b,
20             C_in => ci,
21             C_out => co,
22             Sum => s);
23     a <= '1' after period * 0, '0' after period * 1,
24         '1' after period * 2, '0' after period * 3,
25         '1' after period * 4, '0' after period * 5,
26         '1' after period * 6, '0' after period * 7;
27     b <= '0' after period * 0, '0' after period * 1,
28         '1' after period * 2, '1' after period * 3,
29         '0' after period * 4, '0' after period * 5,
30         '1' after period * 6, '1' after period * 7;
31     ci <= '0' after period * 0, '1' after period * 1,
32         '0' after period * 2, '0' after period * 3,
33         '0' after period * 4, '1' after period * 5,
34         '0' after period * 6, '0' after period * 7;
35 end tb;
```

3 结果

仿真得到的结果如下：

3.1 behavior simulation

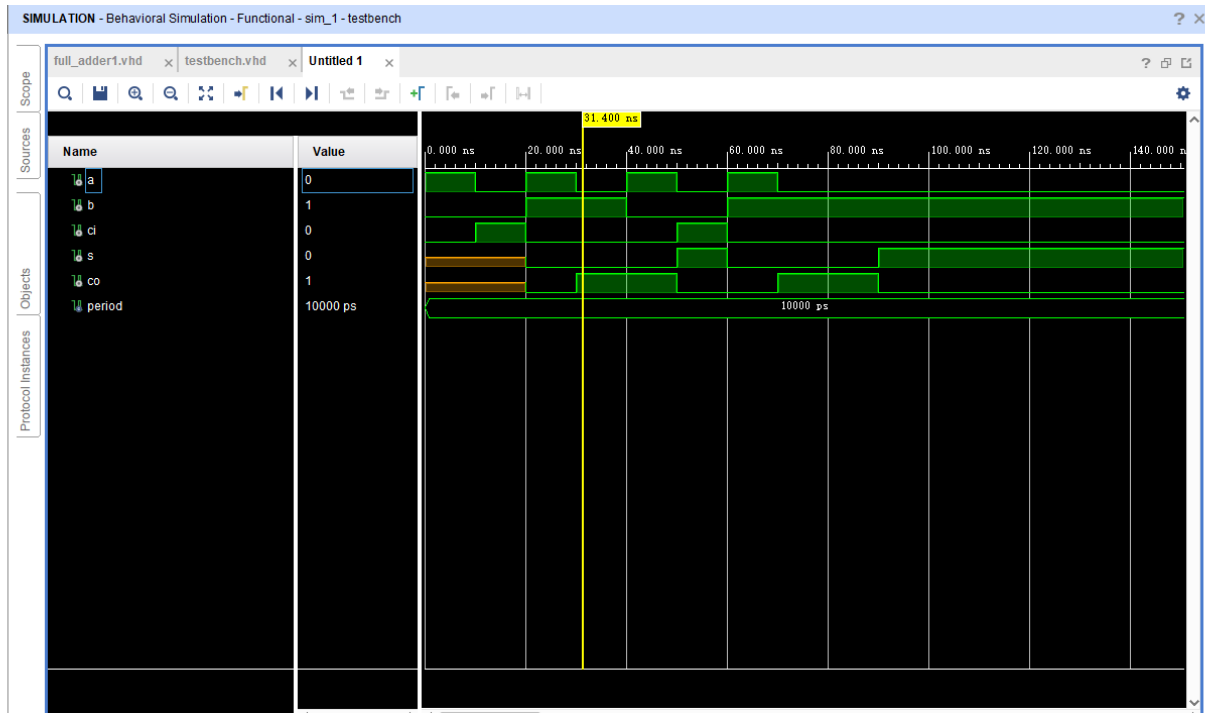


图 2: Behavior Simulation Result

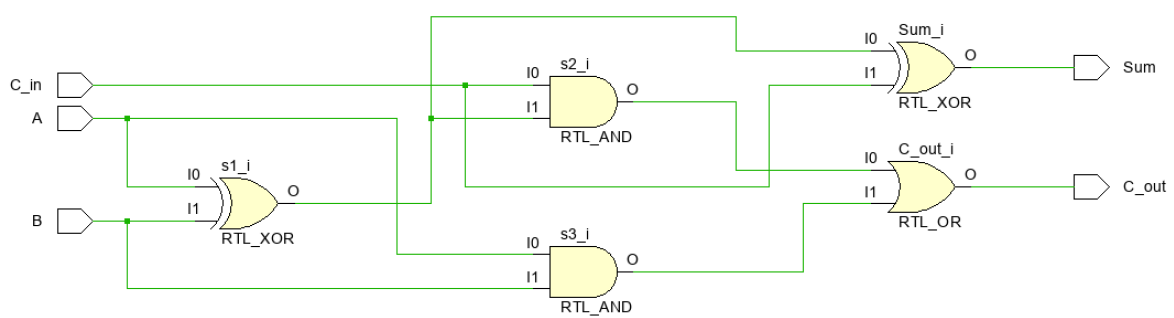


图 3: Behavior Schematic Simulation Result

3.2 post synthesis timing simulation

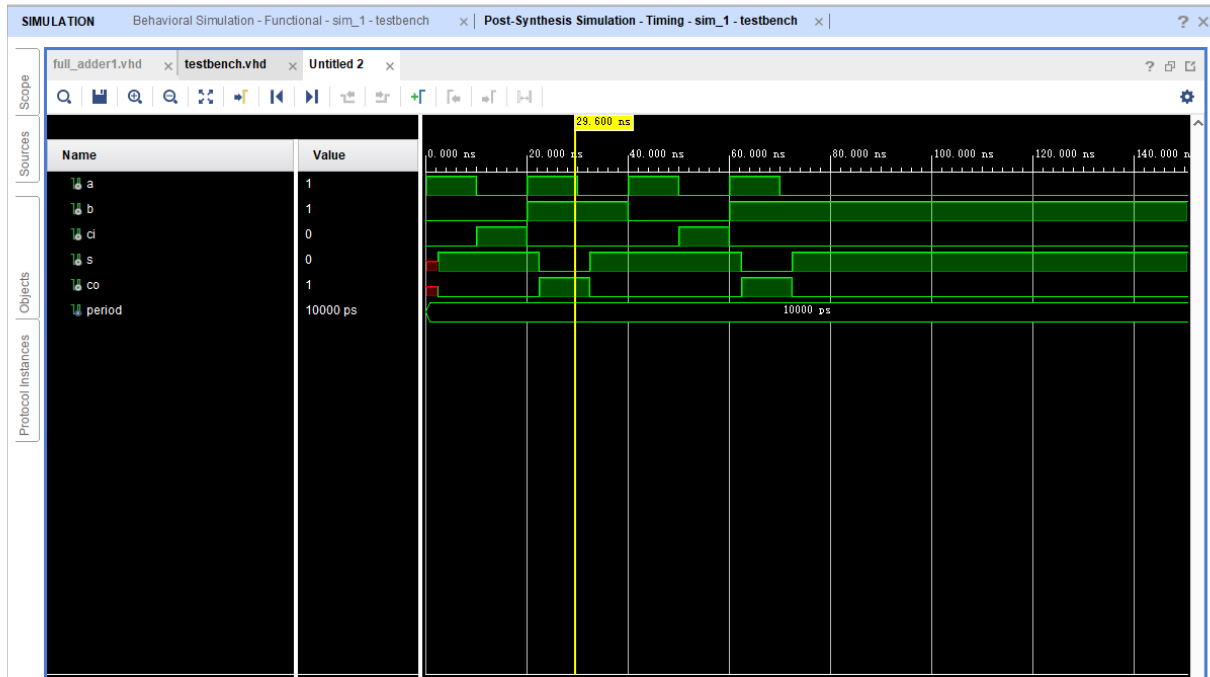


图 4: Post Synthesis Timing Simulation Result

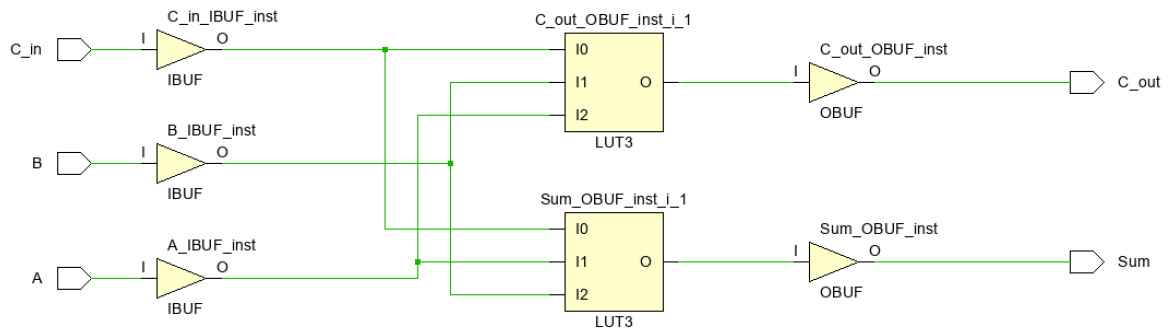


图 5: Synthesis Schematic Simulation Result

3.3 post implementation timing simulation

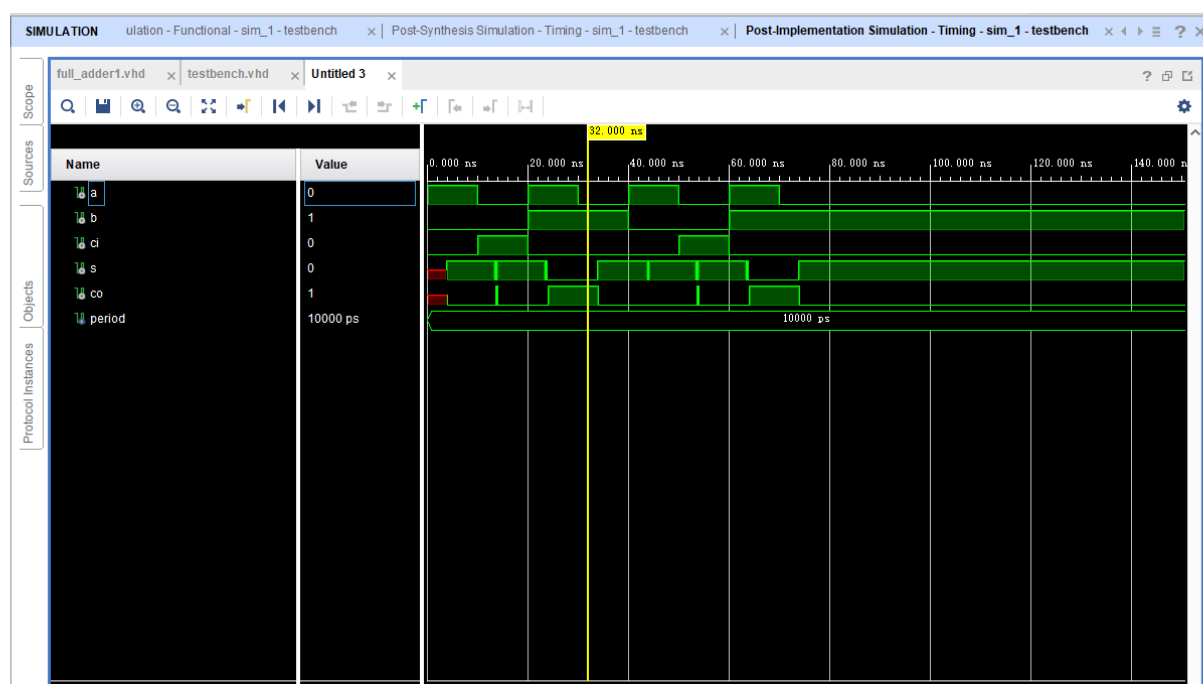


图 6: Post Implementation Timing Simulation Result

4 分析 (Q&A)

4.1 Q1. 仿真结果是否与上页图中的结果一致？

A: 在行为级仿真 (Behavioral Simulation) 中，输出波形与理论参考图高度吻合，完美呈现了我们在 VHDL 代码中人为设定的 10ns 门延迟和累加后的 20ns 级联延迟。然而，在进入综合后时序仿真与实现后时序仿真阶段时，仿真波形与理想参考图发生了显著的偏离。这主要体现在波形不再严格对齐，且响应时间与代码中设定的延迟参数毫无关联。

4.2 Q2. 加法计算结果是否和人们通常的认知一致？

A: 从稳态来看，计算结果与布尔代数真值表完全一致，即全加器在信号稳定后能得出正确的本位和 (Sum) 与进位 (Cout)。但在瞬态或信号翻转的过渡期间，计算结果会短暂地违背人类直觉的十进制或二进制加法认知。此时的波形会出现跳变，处于一种物理上真实存在、但逻辑上看似“错误”的中间状态。

4.3 Q3. 如不一致，导致不一致的原因是什么？

A: 根本原因在于综合工具 (Synthesizer) 对人为延迟代码的剥离以及真实物理延迟的引入。在硬件描述语言中，类似 `after 10 ns` 的语句属于“不可综合 (Unsynthesizable)”语句，仅用于纯逻辑验证，即行为仿真。当代码被转化为真实的硬件电路时，这些理想化约束会被丢弃。时序仿真反映的是真实硅片上的电气特性——信号在晶体管构成的逻辑门中翻转需要时间 (门延迟)，在金属导线中传输也需要时间 (线延迟)，这种纯物理的能量传递过程导致了时间轴上的错位与滞后。

4.4 Q4. 3 种仿真结果有什么不同？

A:

行为级仿真（纯逻辑验证）：波形呈现完美的矩形，边沿绝对陡峭。系统的时序完全被代码中的延迟语法所支配，不包含任何真实的硬件特征。

综合后时序仿真（Post-Synthesis）：逻辑代码已被映射为 FPGA 底层的具体物理单元。由于去除了代码中的人为延迟，此时的波形仅反映了基础的门级延迟（Gate Delay）。响应速度通常极快，且由于电路刚上电尚未稳定，初始状态会出现短暂的未知状态。

实现后时序仿真（Post-Implementation）：最接近芯片流片或下载到开发板后的真实运行状态。在门延迟的基础上，叠加了信号在 FPGA 内部三维空间中实际布线走线所产生的布线延迟（Routing Delay）。波形边缘出现明显的参差不齐，且伴随非理想的杂波。

4.5 Q5. 为什么会有不同？

A: 这是因为这三种仿真所处的硬件模型抽象层级（Abstraction Level）不同。行为仿真是建立在最高抽象层级的数学逻辑模型；综合后仿真下降到了网表（Netlist）层级，开始考虑底层半导体器件的基础电气响应；而实现后仿真则下降到了极低的物理布局层级，其内部包含了由真实电路版图（Layout）带来的寄生电容（Parasitic Capacitance）与寄生电阻（Parasitic Resistance）效应，因此呈现出截然不同的时序特征。

4.6 Q6. 你是否在结果看了一些尖峰？

A: 是的。特别是在最贴近物理真实的实现后时序仿真（Post-Implementation Timing Simulation）中，当输入信号发生跳变时，输出端 Sum 和 Cout 的波形上出现了明显的极窄脉冲信号，即俗称的尖峰或毛刺（Glitch）。

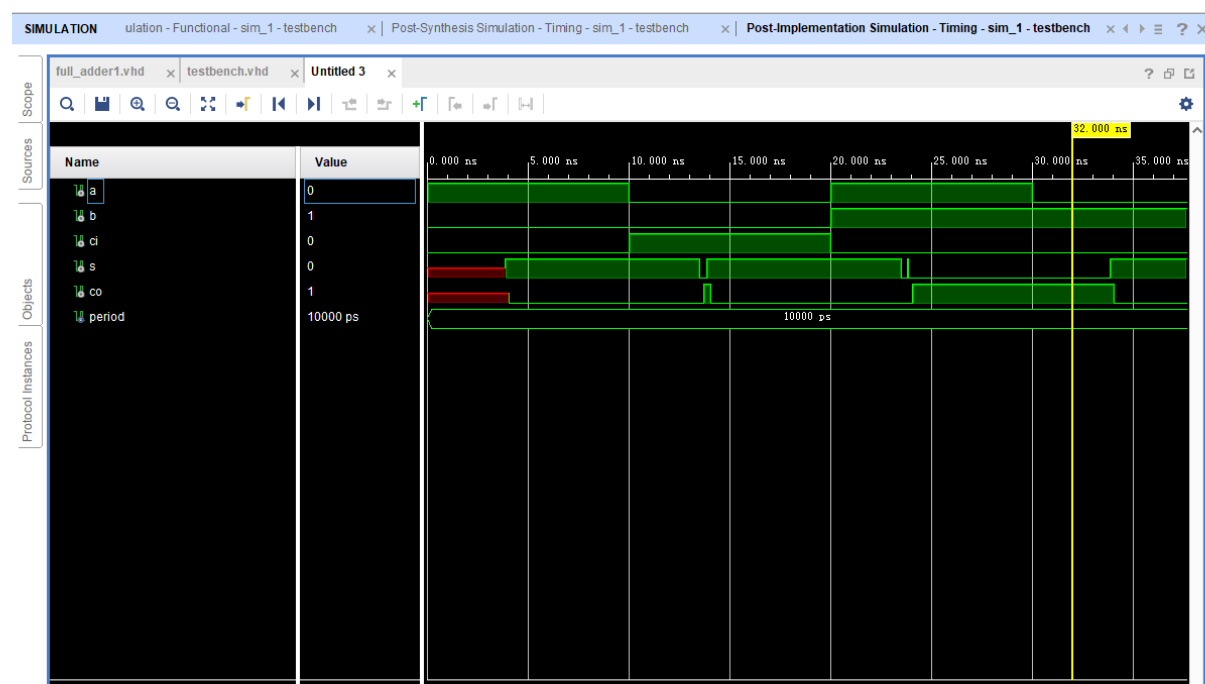


图 7: 出现毛刺的部分

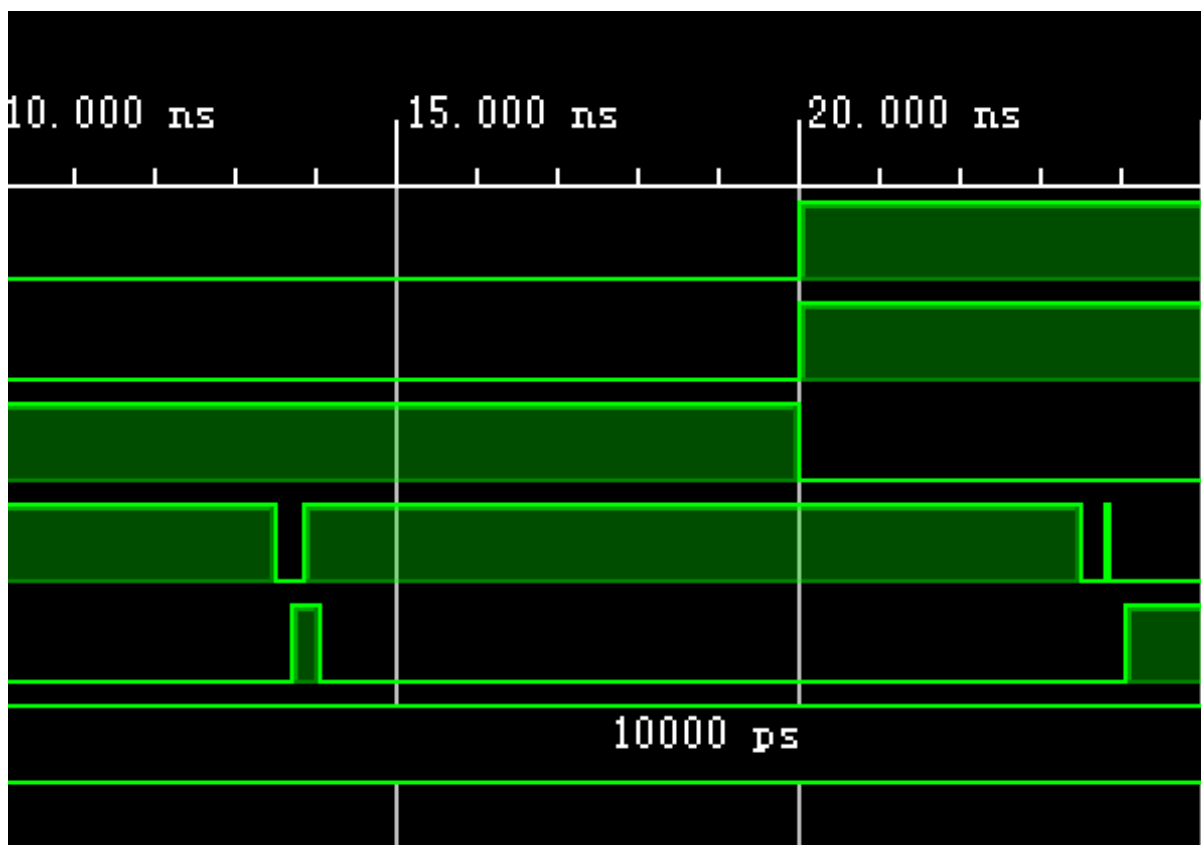


图 8: 毛刺

4.7 Q7. 这些尖峰是由什么引起的？

A: 这些毛刺是数字组合逻辑电路中典型的“竞争与冒险 (Race and Hazard)”现象导致的。在全加器内部，同一个输入信号（例如 A 或 B）会分裂成多路，沿着不同的逻辑门路径向输出端传播。由于每条路径上的逻辑门级数不同，且实际在硅片上连线的长短也不同，导致原本应该同时到达输出端的信号，产生了微小的时间差（路径延迟差）。当这些存在时间差的信号进入最后一级逻辑门时，会在极短的“时间窗口”内满足错误的逻辑触发条件，从而在输出端“挤压”出一个短暂且错误的电压瞬态脉冲。这就是我们在波形上观察到的毛刺现象。

5 结论

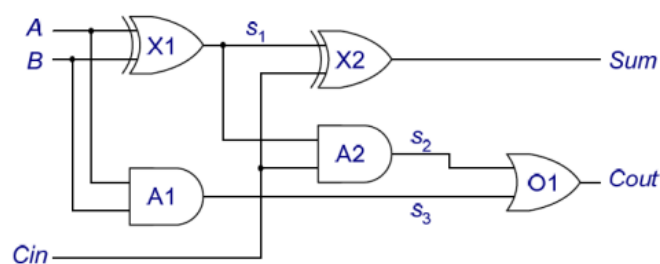


图 9: 全加器结构

本实验成功完成了全加器的 VHDL 设计与 Testbench 验证。该全加器为 1 位全加器且加了人为延时。行为级仿真证实了其在理想情况下的逻辑正确性；而综合后与实现后时序仿真则由于引入了真实的硬件门延迟与布线延迟，出现了预期的信号延迟与毛刺（Glitch）现象。这直观揭示了不同仿真层级间的差异，也强调了在实际数字系统设计中，必须将底层硬件的物理特性纳入考量，以满足最终的时序与功能需求。